

Федеральное государственное автономное образовательное учреждение
высшего образования «Национальный исследовательский университет
ИТМО»

Факультет Программной Инженерии и Компьютерной Техники

Лабораторная работа №5
«Интерполяция функции»
Вариант №2

Группа Р3208

Студент Горин Семён Дмитриевич

Преподаватель Рыбаков Степан Дмитриевич

Содержание

1. Цель работы	3
2. Порядок выполнения работы	3
2.1. Вычислительная реализация задачи	3
2.2. Программная реализация задачи	3
2.2.1. Методы для реализации в программе	4
3. Рабочие формулы	4
3.1. Первая интерполяционная формула Ньютона для интерполирования вперед	4
3.2. Вторая интерполяционная формула Ньютона для интерполирования назад	4
3.3. Первая интерполяционная формула Гаусса ($x > \alpha$)	4
3.4. Вторая интерполяционная формула Гаусса ($x < \alpha$)	5
3.5. Многочлен Лагранжа	5
4. Вычислительная часть	5
5. Листинг программы	6
6. Результаты выполнения программы	15
7. Выводы	16

1. Цель работы

Решить задачу интерполяции, найти значения функции при заданных значениях аргумента, отличных от узловых точек.

2. Порядок выполнения работы

2.1. Вычислительная реализация задачи

X_1	X_2
0,502	0,645

x	y
0,50	1,5320
0,55	2,5356
0,60	3,5406
0,65	4,5462
0,70	5,5504
0,75	6,5559
0,80	7,5594

- 1) Построить таблицу конечных разностей для заданной таблицы. Таблицу отразить в отчете;
- 2) Вычислить значения функции для аргумента X_1 , используя первую или вторую интерполяционную формулу Ньютона. Обратит внимание какой конкретно формулой необходимо воспользоваться;
- 3) Вычислить значения функции для аргумента X_2 , используя первую или вторую интерполяционную формулу Гаусса. Обратит внимание какой конкретно формулой необходимо воспользоваться;
- 4) Вычислить значения функции для аргументов X_1 и X_2 с помощью многочлена Лагранжа;
- 5) Подробные вычисления привести в отчете;

2.2. Программная реализация задачи

- 1) Исходные данные задаются тремя способами:
 - 1) в виде набора данных (таблицы x,y), пользователь вводит значения с клавиатуры;
 - 2) в виде сформированных в файле данных (подготовить не менее трех тестовых вариантов);
 - 3) на основе выбранной функции, из тех, которые предлагает программа, например, $\sin(x)$. Пользователь выбирает уравнение, исследуемый интервал и количество точек на интервале (не менее двух функций).

- 2) Сформировать и вывести таблицу разделенных/конечных разностей (в зависимости от варианта);
- 3) Вычислить приближенное значение функции для заданного значения аргумента, введенного с клавиатуры, указанными методами. Сравнить полученные значения;
- 4) Построить графики заданной функции с отмеченными узлами интерполяции и интерполяционного многочлена Ньютона/Гаусса (разными цветами);
- 5) Программа должна быть протестирована на различных наборах данных, в том числе и некорректных.
- 6) Проанализировать результаты работы программы.
- 7) Реализовать в программе вычисление значения функции для заданного значения аргумента, введенного с клавиатуры, используя схемы Стирлинга;
- 8) Реализовать в программе вычисление значения функции для заданного значения аргумента, введенного с клавиатуры, используя схемы Бесселя;

2.2.1. Методы для реализации в программе

- 1) Многочлен Лагранжа
- 2) Многочлен Ньютона с конечными разностями (1, 2 формулы)
- 3) Многочлен Гаусса (1,2 формулы)

3. Рабочие формулы

3.1. Первая интерполяционная формула Ньютона для интерполирования вперед

$$N_n(x) = y_0 + t\Delta y_0 + \frac{t(t-1)}{2!}\Delta^2 y_0 + \dots + \frac{t(t-1)\dots(t-n+1)}{n!}\Delta^n y_0$$

3.2. Вторая интерполяционная формула Ньютона для интерполирования назад

$$N_n(x) = y_n + t\Delta y_{n-1} + \frac{t(t+1)}{2!}\Delta^2 y_{n-2} + \dots + \frac{t(t+1)\dots(t+n-1)}{n!}\Delta^n y_0$$

3.3. Первая интерполяционная формула Гаусса ($x > \alpha$)

$$\begin{aligned} P_n(x) = & y_0 + t\Delta y_0 + \frac{t(t-1)}{2!}\Delta^2 y_{-1} + \frac{(t+1)t(t-1)}{3!}\Delta^3 y_{-1} + \\ & + \frac{(t+1)t(t-1)(t-2)}{4!}\Delta^4 y_{-2} + \frac{(t+2)(t+1)t(t-1)(t-2)}{5!}\Delta^5 y_{-2} + \\ & + \dots + \frac{(t+n-1)\dots(t-n+1)}{(2n-1)!}\Delta^{2n-1} y_{-n-1} + \frac{(t+n-1)\dots(t-n)}{2n!}\Delta^{2n} y_{-n} \end{aligned}$$

3.4. Вторая интерполяционная формула Гаусса ($x < \alpha$)

$$P_n(x) = y_0 + t\Delta y_0 + \frac{t(t-1)}{2!}\Delta^2 y_{-1} + \frac{(t+1)t(t-1)}{3!}\Delta^3 y_{-1} + \\ + \frac{(t+1)t(t-1)(t-2)}{4!}\Delta^4 y_{-2} + \frac{(t+2)(t+1)t(t-1)(t-2)}{5!}\Delta^5 y_{-2} + \\ + \dots + \frac{(t+n-1)\dots(t-n+1)}{(2n-1)!}\Delta^{2n-1} y_{-n-1} + \frac{(t+n)(t+n-1)\dots(t-n+1)}{2n!}\Delta^{2n} y_{-n}$$

3.5. Многочлен Лагранжа

$$L_n(x) = \sum_{i=0}^n y_i \prod_{j=0, j \neq i}^n \frac{x - x_j}{x_i - x_j}$$

4. Вычислительная часть

x	y	$\Delta^1 y$	$\Delta^2 y$	$\Delta^3 y$	$\Delta^4 y$	$\Delta^5 y$	$\Delta^6 y$
0.5000	1.5320	1.0036	1.0050	1.0056	1.0042	1.0055	1.0035
0.5500	2.5356	0.0014	0.0006	-0.0014	0.0013	-0.0020	
0.6000	3.5406	-0.0008	-0.0020	0.0027	-0.0033		
0.6500	4.5462	-0.0012	0.0047	-0.0060			
0.7000	5.5504	0.0059	-0.0107				
0.7500	6.5559	-0.0166					
0.8000	7.5594						

Таблица 1 — таблица конечных разностей

Так как значение $X_1 = 0,502$ находится вблизи начала таблицы, для него используется первая интерполяционная формула Ньютона, то есть формула интерполирования вперед.

При шаге таблицы $h = 0,05$ получаем

$$t = \frac{X_1 - x_0}{h} = \frac{0,502 - 0,50}{0,05} = 0,04$$

Подставляя конечные разности из таблицы, получаем:

$$N_n(X_1) = 1,5722624855$$

Так как $X_2 = 0,645$ находится левее центрального узла таблицы $x_0 = 0,65$, для него используется вторая интерполяционная формула Гаусса.

При шаге таблицы $h = 0,05$ получаем

$$t = \frac{X_2 - x_0}{h} = \frac{0,645 - 0,65}{0,05} = -0,1$$

Подставляя данные из таблицы, получаем:

$$P_n(X_2) = 4,4457138257$$

Многочлен Лагранжа не требует выбора формулы в зависимости от положения точки интерполяции и применим для любых расположений узлов.

Подставив значения из таблицы, получаем:

$$L_n(X_1) = 1,5722624855$$

$$L_n(X_2) = 4,4457138257$$

Значение многочлена Лагранжа совпадает со значениями, полученными методами Ньютона и Гаусса, что подтверждает корректность вычислений.

5. Листинг программы

```

1: package lab.math;
2:
3: import lab.model.DataPoint;
4: import lab.model.DataSet;
5:
6: import java.util.ArrayList;
7: import java.util.List;
8:
9: public final class InterpolationService {
10:     private static final double SPACING_TOLERANCE = 1e-9;
11:
12:     public List<MethodComputation> computeAll(DataSet dataSet, double
x) {
13:         List<MethodComputation> computations = new ArrayList<>();
14:         computations.add(new MethodComputation("Lagrange",
interpolateLagrange(dataSet.points(), x), ""));
15:
16:         if (canUseNewton(dataSet)) {
17:             boolean useForward = x <= midpoint(dataSet);
18:             computations.add(new MethodComputation(
19:                 useForward ? "Newton forward" : "Newton backward",
20:                 interpolateNewton(dataSet, x),
21:                 useForward ? "first formula" : "second formula"
22:             ));
23:         } else {
24:             computations.add(new MethodComputation("Newton", null,
"nodes are not equally spaced"));
25:         }
26:
27:         if (canUseGauss(dataSet)) {
28:             boolean useFirstFormula = x >= centralNode(dataSet).x();
29:             computations.add(new MethodComputation(
30:                 useFirstFormula ? "Gauss first" : "Gauss second",
31:                 interpolateGauss(dataSet, x),
32:                 useFirstFormula ? "x >= center" : "x < center"
33:             ));
34:         } else {

```

```

35:         computations.add(new MethodComputation("Gauss", null,
"nodes are not equally spaced or there are fewer than 3 points"));
36:     }
37:
38:     if (canUseStirling(dataSet)) {
39:         computations.add(new MethodComputation("Stirling",
interpolateStirling(dataSet, x), "centered formula"));
40:     } else {
41:         computations.add(new MethodComputation("Stirling", null,
"nodes are not equally spaced or there are fewer than 3 points"));
42:     }
43:
44:     if (canUseBessel(dataSet)) {
45:         computations.add(new MethodComputation("Bessel",
interpolateBessel(dataSet, x), "central pair formula"));
46:     } else {
47:         computations.add(new MethodComputation("Bessel", null,
"nodes are not equally spaced or there are fewer than 4 points"));
48:     }
49:
50:     return computations;
51: }
52:
53: public double interpolateLagrange(List<DataPoint> points, double
x) {
54:     double result = 0.0;
55:     for (int i = 0; i < points.size(); i++) {
56:         DataPoint current = points.get(i);
57:         double basis = 1.0;
58:         for (int j = 0; j < points.size(); j++) {
59:             if (i == j) {
60:                 continue;
61:             }
62:             basis *= (x - points.get(j).x()) / (current.x() -
points.get(j).x());
63:         }
64:         result += current.y() * basis;
65:     }
66:     return result;
67: }
68:
69: public double interpolateNewton(DataSet dataSet, double x) {
70:     List<DataPoint> points = dataSet.points();
71:     if (!canUseNewton(dataSet)) {
72:         throw new IllegalArgumentException("Newton interpolation
requires equally spaced nodes");
73:     }
74:
75:     FiniteDifferenceTable table = new
FiniteDifferenceTable(points);
76:     double step = spacing(points);
77:     double origin = points.get(0).x();

```

```

78:         double target = points.get(points.size() - 1).x();
79:         boolean useForward = x <= midpoint(dataSet);
80:
81:         if (useForward) {
82:             double t = (x - origin) / step;
83:             double result = points.get(0).y();
84:             double factor = 1.0;
85:             for (int order = 1; order < points.size(); order++) {
86:                 factor *= t - (order - 1);
87:                 result += factor * table.difference(0, order) /
factorial(order);
88:             }
89:             return result;
90:         }
91:
92:         double t = (x - target) / step;
93:         double result = points.get(points.size() - 1).y();
94:         double factor = 1.0;
95:         for (int order = 1; order < points.size(); order++) {
96:             factor *= t + (order - 1);
97:             result += factor * table.difference(points.size() - 1 -
order, order) / factorial(order);
98:         }
99:         return result;
100:     }
101:
102:     public double interpolateGauss(DataSet dataSet, double x) {
103:         DataSet selected = selectOddWindow(dataSet, x);
104:         List<DataPoint> points = selected.points();
105:         if (!canUseGauss(dataSet)) {
106:             throw new IllegalArgumentException("Gauss interpolation
requires equally spaced nodes and at least 3 points");
107:         }
108:
109:         FiniteDifferenceTable table = new
FiniteDifferenceTable(points);
110:         double step = spacing(points);
111:         int centerIndex = points.size() / 2;
112:         double centerX = points.get(centerIndex).x();
113:         double t = (x - centerX) / step;
114:         boolean useFirstFormula = x >= centerX;
115:
116:         double result = points.get(centerIndex).y();
117:         double factor = 1.0;
118:         int maxOrder = points.size() - 1;
119:
120:         for (int order = 1; order <= maxOrder; order++) {
121:             factor *= useFirstFormula ? gaussFirstFactor(order, t) :
gaussSecondFactor(order, t);
122:             int differenceIndex = useFirstFormula ? centerIndex -
(order / 2) : centerIndex - ((order + 1) / 2);
123:             if (differenceIndex < 0 || differenceIndex + order >=
points.size()) {

```



```

124:             break;
125:         }
126:         result += factor * table.difference(differenceIndex,
order) / factorial(order);
127:     }
128:     return result;
129: }
130:
131: public double interpolateStirling(DataSet dataSet, double x) {
132:     List<DataPoint> points = selectCenteredOddWindow(dataSet,
x).points();
133:     if (!canUseStirling(dataSet)) {
134:         throw new IllegalArgumentException("Stirling interpolation
requires equally spaced nodes and at least 3 points");
135:     }
136:     return interpolateStirling(points, x);
137: }
138:
139: public double interpolateBessel(DataSet dataSet, double x) {
140:     List<DataPoint> points = selectCenteredEvenWindow(dataSet,
x).points();
141:     if (!canUseBessel(dataSet)) {
142:         throw new IllegalArgumentException("Bessel interpolation
requires equally spaced nodes and at least 4 points");
143:     }
144:
145:     if (points.size() < 4) {
146:         throw new IllegalArgumentException("Bessel interpolation
requires at least 4 points");
147:     }
148:
149:     double left = interpolateStirling(points.subList(0,
points.size() - 1), x);
150:     double right = interpolateStirling(points.subList(1,
points.size()), x);
151:     return (left + right) / 2.0;
152: }
153:
154: public boolean canUseNewton(DataSet dataSet) {
155:     return dataSet.points().size() >= 2 &&
isEquallySpaced(dataSet.points());
156: }
157:
158: public boolean canUseGauss(DataSet dataSet) {
159:     return dataSet.points().size() >= 3 &&
isEquallySpaced(dataSet.points());
160: }
161:
162: public boolean canUseStirling(DataSet dataSet) {
163:     return dataSet.points().size() >= 3 &&
isEquallySpaced(dataSet.points());
164: }

```

```

165:
166:     public boolean canUseBessel(DataSet dataSet) {
167:         return dataSet.points().size() >= 4 &&
isEquallySpaced(dataSet.points());
168:     }
169:
170:     private boolean isEquallySpaced(List<DataPoint> points) {
171:         double step = spacing(points);
172:         for (int index = 2; index < points.size(); index++) {
173:             double currentStep = points.get(index).x() -
points.get(index - 1).x();
174:             if (Math.abs(currentStep - step) > SPACING_TOLERANCE) {
175:                 return false;
176:             }
177:         }
178:         return true;
179:     }
180:
181:     private double spacing(List<DataPoint> points) {
182:         if (points.size() < 2) {
183:             throw new IllegalArgumentException("At least two points
are required");
184:         }
185:         return points.get(1).x() - points.get(0).x();
186:     }
187:
188:     private double midpoint(DataSet dataSet) {
189:         List<DataPoint> points = dataSet.points();
190:         return (points.get(0).x() + points.get(points.size() -
1).x()) / 2.0;
191:     }
192:
193:     private DataPoint centralNode(DataSet dataSet) {
194:         List<DataPoint> points = selectOddWindow(dataSet,
midpoint(dataSet)).points();
195:         return points.get(points.size() / 2);
196:     }
197:
198:     private DataSet selectCenteredOddWindow(DataSet dataSet, double x)
{
199:         List<DataPoint> points = dataSet.points();
200:         int windowSize = points.size() % 2 == 1 ? points.size() :
points.size() - 1;
201:         if (windowSize < 3) {
202:             throw new IllegalArgumentException("Stirling interpolation
requires at least 3 points");
203:         }
204:         return selectWindow(dataSet, x, windowSize);
205:     }
206:
207:     private DataSet selectCenteredEvenWindow(DataSet dataSet, double
x) {

```

```

208:         List<DataPoint> points = dataSet.points();
209:         int windowSize = points.size() % 2 == 0 ? points.size() :
points.size() - 1;
210:         if (windowSize < 4) {
211:             throw new IllegalArgumentException("Bessel interpolation
requires at least 4 points");
212:         }
213:         return selectWindow(dataSet, x, windowSize);
214:     }
215:
216:     private DataSet selectOddWindow(DataSet dataSet, double x) {
217:         List<DataPoint> points = dataSet.points();
218:         if (points.size() % 2 == 1) {
219:             return dataSet;
220:         }
221:
222:         int windowSize = points.size() - 1;
223:         if (windowSize < 3) {
224:             throw new IllegalArgumentException("Gauss interpolation
requires at least 3 points");
225:         }
226:
227:         return selectWindow(dataSet, x, windowSize);
228:     }
229:
230:     private DataSet selectWindow(DataSet dataSet, double x, int
windowSize) {
231:         List<DataPoint> points = dataSet.points();
232:
233:         int closestIndex = 0;
234:         double closestDistance = Math.abs(points.get(0).x() - x);
235:         for (int index = 1; index < points.size(); index++) {
236:             double distance = Math.abs(points.get(index).x() - x);
237:             if (distance < closestDistance) {
238:                 closestDistance = distance;
239:                 closestIndex = index;
240:             }
241:         }
242:
243:         int start = Math.max(0, Math.min(closestIndex - windowSize /
2, points.size() - windowSize));
244:         return DataSet.table(dataSet.name() + " (window)",
points.subList(start, start + windowSize));
245:     }
246:
247:     private double interpolateStirling(List<DataPoint> points, double
x) {
248:         FiniteDifferenceTable table = new
FiniteDifferenceTable(points);
249:         double step = spacing(points);
250:         int centerIndex = points.size() / 2;
251:         double centerX = points.get(centerIndex).x();
252:         double t = (x - centerX) / step;

```

```

253:
254:     double result = points.get(centerIndex).y();
255:     for (int order = 1; order < points.size(); order++) {
256:         if (order == 1) {
257:             result += t * averageDifference(table, centerIndex -
1, order) / factorial(order);
258:             continue;
259:         }
260:
261:         if (order % 2 == 0) {
262:             double factor = stirlingEvenFactor(order, t);
263:             int differenceIndex = centerIndex - (order / 2);
264:             result += factor * table.difference(differenceIndex,
order) / factorial(order);
265:         } else {
266:             double factor = stirlingOddFactor(order, t);
267:             int differenceIndex = centerIndex - ((order + 1) / 2);
268:             result += factor * averageDifference(table,
differenceIndex, order) / factorial(order);
269:         }
270:     }
271:
272:     return result;
273: }
274:
275: private double interpolateBessel(List<DataPoint> points, double x)
{
276:     FiniteDifferenceTable table = new
FiniteDifferenceTable(points);
277:     double step = spacing(points);
278:     int leftCenterIndex = points.size() / 2 - 1;
279:     double centerX = points.get(leftCenterIndex).x();
280:     double t = (x - centerX) / step;
281:
282:     double result = (points.get(leftCenterIndex).y() +
points.get(leftCenterIndex + 1).y()) / 2.0;
283:     for (int order = 1; order < points.size(); order++) {
284:         if (order == 1) {
285:             result += (t - 0.5) *
table.difference(leftCenterIndex, order) / factorial(order);
286:             continue;
287:         }
288:
289:         if (order % 2 == 0) {
290:             double factor = besselEvenFactor(order, t);
291:             int differenceIndex = leftCenterIndex - (order / 2);
292:             result += factor * averageDifference(table,
differenceIndex, order) / factorial(order);
293:         } else {
294:             double factor = besselOddFactor(order, t);
295:             int differenceIndex = leftCenterIndex - ((order - 1) /
2);

```

```

296:         result += factor * table.difference(differenceIndex,
order) / factorial(order);
297:     }
298: }
299:
300:     return result;
301: }
302:
303:     private double averageDifference(FiniteDifferenceTable table, int
index, int order) {
304:         return (table.difference(index, order) +
table.difference(index + 1, order)) / 2.0;
305:     }
306:
307:     private double stirlingOddFactor(int order, double t) {
308:         double factor = t;
309:         for (int i = 1; i <= (order - 1) / 2; i++) {
310:             factor *= t * t - i * i;
311:         }
312:         return factor;
313:     }
314:
315:     private double stirlingEvenFactor(int order, double t) {
316:         double factor = t * t;
317:         for (int i = 1; i < order / 2; i++) {
318:             factor *= t * t - i * i;
319:         }
320:         return factor;
321:     }
322:
323:     private double besselOddFactor(int order, double t) {
324:         double factor = t - 0.5;
325:         for (int i = 1; i <= (order - 1) / 2; i++) {
326:             factor *= (t + i - 1) * (t - i);
327:         }
328:         return factor;
329:     }
330:
331:     private double besselEvenFactor(int order, double t) {
332:         double factor = t * (t - 1.0);
333:         for (int i = 1; i < order / 2; i++) {
334:             factor *= (t + i) * (t - i - 1.0);
335:         }
336:         return factor;
337:     }
338:
339:     private double gaussFirstFactor(int order, double t) {
340:         if (order == 1) {
341:             return t;
342:         }
343:         return order % 2 == 0 ? t - (order / 2.0) : t + ((order - 1) /
2.0);
344:     }

```

```

345:
346:     private double gaussSecondFactor(int order, double t) {
347:         if (order == 1) {
348:             return t;
349:         }
350:         return order % 2 == 0 ? t + (order / 2.0) : t - ((order - 1) /
2.0);
351:     }
352:
353:     private double factorial(int value) {
354:         double result = 1.0;
355:         for (int index = 2; index <= value; index++) {
356:             result *= index;
357:         }
358:         return result;
359:     }
360: }
361:

```

6. Результаты выполнения программы

```
1: 1 - enter table manually
2: 2 - load table from file
3: 3 - generate table from function
4: 0 - exit
5: Choose an option: 2
6: Available files:
7: 1 - sample_table_1.json
8: 2 - sample_table_2.json
9: 3 - sample_table_3.json
10: 4 - uneven_sine_uneven_nodes.json
11: Choose a file: 2
12: Enter interpolation argument x: 0.502
13:
14: Data set: Table 1.2
15: Finite differences table
16:      x          y          Δ1y          Δ2y
    Δ3y          Δ4y          Δ5y          Δ6y
17:    0.500000    1.532000    1.003600    1.005000
    1.005600    1.004200    1.005500    1.003500
18:    0.550000    2.535600    0.001400    0.000600
    -0.001400    0.001300    -0.002000
19:    0.600000    3.540600    -0.000800    -0.002000
    0.002700    -0.003300
20:    0.650000    4.546200    -0.001200    0.004700
    -0.006000
21:    0.700000    5.550400    0.005900    -0.010700
22:    0.750000    6.555900    -0.016600
23:    0.800000    7.559400
24:
25: Lagrange: 1.5722624855
26: Newton forward: 1.5722624855 [first formula]
27: Gauss second: 1.5722624855 [x < center]
28: Stirling: 1.5722624855 [centered formula]
29: Bessel: 1.5748291955 [central pair formula]
30: Chart saved to: /home/astrosoup/computational-mathematics-lab-5/build/
    charts/table_1_2.png
31:
```

Листинг 1 — Вывод программы в консоль

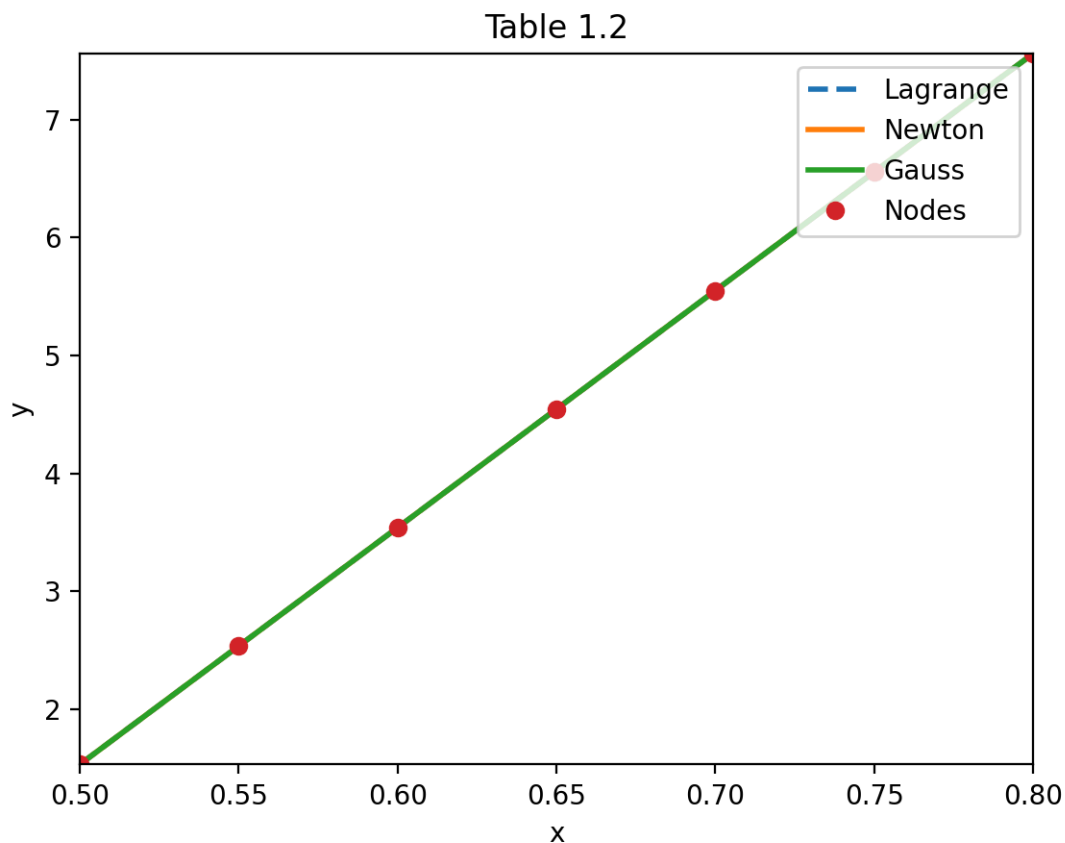


Рис. 1 — Полученный в результате работы график

7. Выводы

По итогу выполнения лабораторной работы были реализованы алгоритмы интерполяции функции с помощью многочлена Лагранжа, многочлена Ньютона с конечными разностями и многочлена Гаусса. Результаты, полученные этими методами, совпали, что подтверждает корректность реализации. Также были реализованы схемы Стирлинга и Бесселя, которые также дали результаты, близкие к основным методам.